

**Advanced Data Management
2021/2022**

Q3. (up to 10 lines) [**only for COMPUTER ENGINEERING STUDENTS**]

Compare single leader and multi-leader replication approaches.

Q2. (up to 10 lines)

Describe the distributed hashing approach used by Riak and Cassandra.

Q3. (up to 10 lines)

Compare the approach for representing associations (relationships) in the graph-based data model and in the relational model.

Q4. (up to 10 lines) [**only for COMPUTER SCIENCE STUDENTS**]

Discuss the main differences among RDF, RDFS, and OWL.

Exercise 1 [NoSQL aggregate-oriented data stores]

Consider the following Cassandra table:

```
CREATE TABLE books_by_author (
  author_id uuid,
  release_date timestamp,
  book_id uuid,
  author_name text,
  book_title text
  PRIMARY KEY ((author_id), release_date, book_id)
);
```

1. Write one query (in **natural language**) that can be executed on table `books_by_author`, without creating any index and without the need of specifying the “ALLOW FILTERING” clause. The query must contain: (i) selection conditions over at least two different attributes; (ii) at least one range condition.

Determine the books written by the author with id equal to 3 after January 2019.

2. Discuss whether the request
Check if the book with id = 165 has been released after year 2017 and, if yes, return the names of its authors
 can be executed on table `books_by_author`, without creating any index and without the need of specifying the “ALLOW FILTERING” clause.

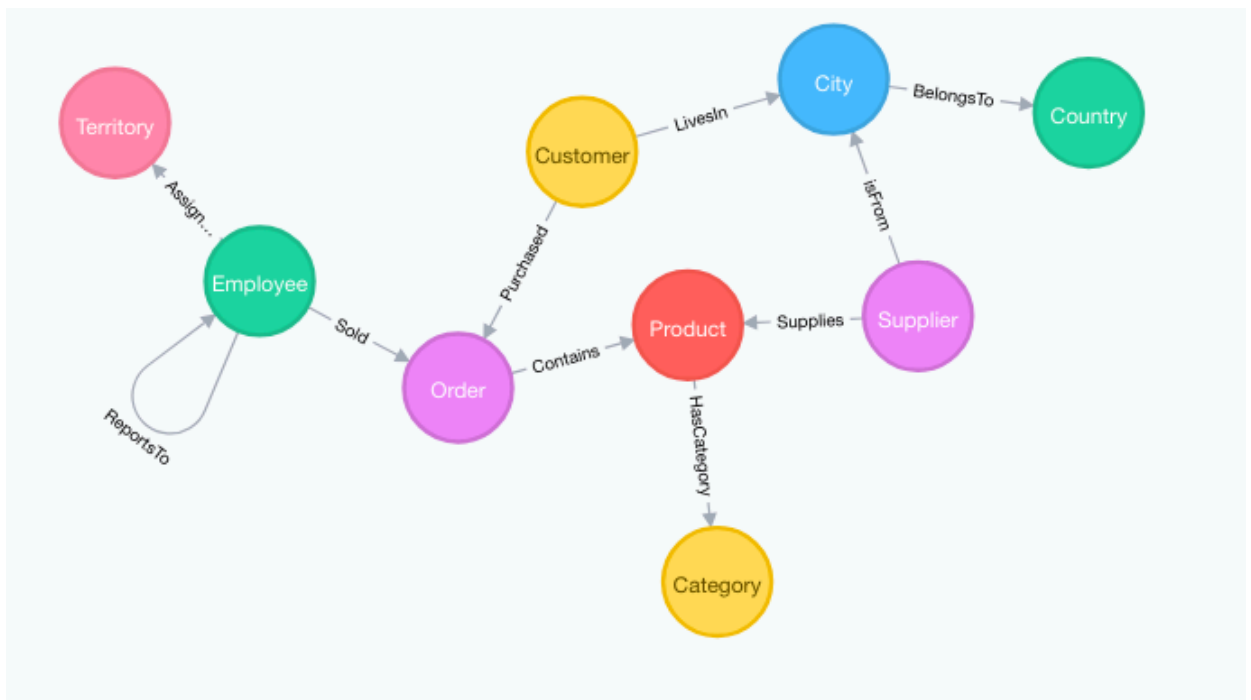
It cannot be executed because it does not specify any equality condition on the partition key (author_id).

3. If the answer to the previous question is “no”, design a new table such that the previous request can be executed on it, without creating any index and without the need of specifying the “ALLOW FILTERING” clause.

```
CREATE TABLE books_by_author (
  author_id uuid,
  release_date timestamp,
  book_id uuid,
  author_name text,
  book_title text
  PRIMARY KEY ((book_id), release_date, author_id)
);
```

Exercise 2 [NoSQL graph-based data stores]

Consider the following graph schema:



Write in **natural language** and in **Cypher** one **single** request over the graph above such that:

- it involves at least four node types;
- it involves at least three relationship types;
- it contains at least one aggregate function;
- it contains at least one condition over a relationship property, at your choice;
- it contains at least one condition over a node property, at your choice.

Determine the number of orders purchased in 2021 by customers that are at least 70 years old including a product of category “electronic devices”

[Assuming that the purchase date is an attribute of relationship “Purchased”, the age is an attribute of Customers, and type is an attribute of “Category”]

```

MATCH (:Category) - (:HasCategory {type = "electronic devices"}) - (:Product {}) -
(:Contains) - (n:Order) - (p:Purchased) - (c:Customer)
WHERE c.age > 70 AND p.date >= "1/1/2021 AND p.date <= 31/12/2021"
RETURN count(n)
  
```

Exercise 3 [Semantic data stores] [only for COMPUTER SCIENCE STUDENTS]

The following is an RDF dataset about courses at the university of Stuttgart and elsewhere.

```

ex:Course owl:disjointWith ex:LanguageClass .
ex:Polish2 rdf:type ex:LanguageClass .
ex:Polish2 ex:hasName "Polish 2 (level A1.2)" .
ex:Polish2 ex:takesPlaceInSemester ex:WS2015 .
ex:Polish2 ex:takesPlaceAt ex:Sprachenzentrum .
ex:SemWeb15w rdf:type ex:Course .
ex:SemWeb15w ex:hasName "Semantic Web" .
ex:SemWeb15w ex:hasECTS 3 .
ex:SemWeb15w ex:takesPlaceInSemester ex:WS2015 .
ex:SemWeb15w ex:takesPlaceInRoom ex:V5.01 .
ex:MethodsInCL15w rdf:type ex:Course .
ex:MethodsInCL15w ex:hasName "Methods in Computational Linguistics" .
ex:MethodsInCL15w ex:hasECTS 9 .
ex:MethodsInCL15w ex:hasSWS 6 .
ex:MethodsInCL15w ex:takesPlaceInSemester ex:WS2015 .
ex:MethodsInCL15w ex:takesPlaceInRoom ex:V5.01 .
ex:AdvancedSemantics16s rdf:type ex:Course .
ex:AdvancedSemantics16s ex:hasName "Advanced Semantics" .
ex:AdvancedSemantics16s ex:hasSWS 2 .
ex:AdvancedSemantics16s ex:takesPlaceInSemester ex:SS2016 .
ex:AdvancedSemantics16s ex:takesPlaceInRoom ex:M12.01 .
ex:MachineLearning4 rdf:type ex:MOOC .
ex:MachineLearning4 ex:hasName "Machine Learning" .
ex:MachineLearning4 ex:takesPlaceOnPlatform ex:Coursera .
ex:V5.01 rdf:type ex:LectureHall .
ex:V5.01 ex:isLocatedInBuilding ex:Pfaffenwaldring5b .
ex:V5.01 ex:maxPeople 24 .
ex:Pfaffenwaldring5b ex:isLocatedOnCampus ex:CampusVaihingen .
ex:M12.01 rdf:type ex:LectureHall .
ex:M12.01 ex:isLocatedInBuilding ex:Azenbergstr12 .
ex:M12.01 ex:maxPeople 100 .
ex:Azenbergstr12 ex:isLocatedOnCampus ex:CampusStadtmitte .

```

Write a SPARQL query for each of the following requests:

1. The URI, name and semester in which the course takes places of all courses [note: the term “course” here and in the following does not include MOOCs or language classes].

```

SELECT ?c, ?n, ?s
WHERE {
  ?c rdf:type ex:Course .
  ?c :hasName ?n .

```

```
?c : takesPlaceInSemester ?s .
}
```

2. The URI, the number of ECTS (if available) and the number of SWS, i.e., Semester Weekly Hours, (if available) of all courses.

```
SELECT ?c, ?e, ?s
WHERE {
  ?c rdf:type ex:Course .
  OPTIONAL{?c :hasECTS ?e .}
  OPTIONAL{?c : takesSWS ?s .}
}
```

The URI and location (room resp. platform) of all courses that are Massive Open Online Courses (MOOCs) and all courses that take place either in Pfaffen- waldring 5b or Azenbergstr. 12.

```
SELECT ?c, ?r
WHERE {
  {
    ?c rdf:type ex:Course .
    ?c ex:takesPlaceInRoom ?r .
  }
  UNION
  {
    ?c rdf:type ex:MOOC .
    ?c ex:takesPlaceOnPlatform ?r .
  }
}
```